Segmentation of Bangla Words in Scene Images

Prakriti Banik*
Indian Statistical Institute
Kolkata 700108
prakriti.banik@gmail.com

Ujjwal Bhattacharya
Indian Statistical Institute
Kolkata 700108
ujjwal@iscal.ac.in

Swapan K. Parui
Indian Statistical Institute
Kolkata 700108
swapan@iscal.ac.in

ABSTRACT

Some studies on extraction of *Bangla* texts from scene images are available in the literature. Also, OCR of printed *Bangla* texts has been extensively studied. However, the performance of available *Bangla* OCR on scene texts is not acceptable. In this article, we present our recent study of segmentation of characters or their parts from *Bangla* texts extracted from scene images. The proposed approach detects the background and text by a combination of two algorithms: unsupervised learning algorithm K-means clustering and Otsu's threshold selection. We propose a criterion to choose an optimal K value for K-means clustering. The text segmentation is based on region growing and extraction of both headline and baseline of such texts. These two lines divide a *Bangla* word into three horizontal zones. The present algorithm segments characters or their parts in each individual zone. This zone-based segmentation approach helps to reduce the number of symbols to be handled by the classifier in the next stage of the OCR system. Our algorithm can also detect an image having only numerals, avoiding zone detection in that case. Extracted scene texts are often affected by artifacts and our segmentation algorithm can remove them efficiently. Our algorithm has been tested on 2460 *Bangla* words extracted from 260 scene images.

Keywords

K-means clustering, RGB normalization, character segmentation, baseline detection

1. INTRODUCTION

Character segmentation is one of the important modules of a complete optical character recognition (OCR) system. There have been several studies on OCR of *Bangla* script. However, as indicated in the paper [13], a natural image text OCR is far more complex than OCR in scanned documents. The algorithms developed for printed *Bangla* OCR may not always work for scene text recognition. This is due to multiple degradations such as low resolution, varying background and foreground colors, uneven lighting, perspective etc. Further, in the case of texts embedded in such natural scene images, additional complications arise from the use of non-formatted texts and artistic fonts, like the ones used in roadside hoardings. Moreover, a single word sometime contains varying font sizes, designs, colors and lightness. These features make character segmentation of such texts a challenging task and often it is poorly handled by existing OCR algorithms. The problem is more complex for scripts like *Bangla* in which the characters of a word are topologically connected due to their headline (*matra*). Additional variability occurs due to the fact that there are a few characters of *Bangla* alphabet, which do not get attached with the headline. The primary focus of research on camera captured scene images has been on detection, localization and extraction of texts embedded in such images. Although related studies [6, 8, 15] are primarily restricted to English and a few other scripts of the developed countries, the same problem has recently been studied for two major Indian scripts *Bangla* and *Devanagari* [1, 4, 7]. Two related surveys can be found in [10, 11].

Holistic approach to recognize *Bangla* texts instead of segmenting each character or its part separately is a feasible option for printed documents as the availability of test data is abundant. But for scene images this is not considered as a proper approach as the scope of collecting the amount of data needed is less. Therefore segmenting each character or its part is necessary and proper for the recognition phase. Character segmentation of scanned printed texts had been studied since early days of document analysis research and an early survey can be found in [2]. Segmentation of *Bangla* texts has been studied in [3]. These schemes often fail to segment characters of scene texts. In a recent study [12], a Log-Gabor filter has been used to segment characters from images of scene texts. However, this approach also cannot handle word images in which characters are topologically connected by a headline. In the present study, we focus on the segmentation of scene word which is already detected and localized from a larger image. Here, we distinguish the text and the background color by using two methods. Firstly, we apply unsupervised K-means clustering algorithm in the RGB color space followed by RGB normalization. Normalization is executed to remove the effect of lightness variation in an image which is a very common feature in scene images. Secondly, at the same

*Corresponding author

ICVGIP '12, December 16-19, 2012, Mumbai, India

time the pixels having similar or almost similar R, G, B values are processed without normalization through Otsu's binarization. We then correct the skewness of text and detect the headline of *Bangla* words. We also compute the baseline of these words. Based on the above two lines we obtain three zones of *Bangla* words as shown in Fig. 1. In each zone, we obtain individual characters or their parts separately. For numerals we avoid headline and baseline detection.

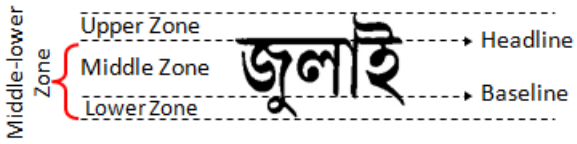


Figure 1: Zones of *Bangla* script.

There has not been much work in extraction and recognition of *Bangla* text parts that may be present in scene images. However, to develop a recognition scheme, one needs to have a database annotated at word and character levels. Since there is no such database in *Bangla*, we have developed a database of 2460 *Bangla* word images extracted from 260 scene images. These word images have been manually annotated at the word level. Now, since it is difficult to segment a word image into ideal characters/modifiers, we have developed a segmentation algorithm that segments a word image into symbols that are either a numeral or a character or part of a character or a modifier or part of a modifier. As a result, a symbol level annotation becomes possible. This can finally be useful in character level recognition.

Our proposed algorithm works in five steps: text and background segmentation, headline detection, character segmentation, baseline detection and segment unification. A combination of Otsu's global thresholding and K-means clustering separates the text from background. First, for a selected set of pixels we perform Otsu's thresholding and for the complementary set we apply K-means clustering iteratively for different values of K. In the present paper, Otsu's algorithm is employed only on a limited number of pixels in the input image : the pixels whose R, G, B values are very close to one another (ie, pixels that are nearly gray). Thus, fine tuning of this algorithm may not be necessary in the present case. We use normalized RGB values for K-means clustering to remove effect of lightness. Then a selection criterion is applied to select the optimum K value. As our input image is a bounding box of a *Bangla* text, we expect to have at most 4 to 5 clusters to be present in the image. The cluster containing the maximum number of border and penultimate border pixels is defined as the background cluster. And by computing some features, such as, the cluster with maximum distance or minimum overlap from the background cluster, thickness of cluster and thickness uniformity of each cluster (for all the clusters obtained from Otsu's binarization and K-means clustering) we determine the text cluster. This process naturally filters out text background artifacts and overlapping strokes with no extra layer of processing. After text and background separation, headlines are detected using projection profile computed by row-wise sum of gray values for each row in the upper half of word image. If no headline is detected then we consider that the image contains numerals or texts without any headline. In the next step, we apply region growing algorithm with some stopping criteria defined by us, which leads to successful segmentation of text symbols from the upper and middle-lower zones. In the fourth step, the algorithm detects the baseline by analyzing the height of each segment from the middle-lower zones. This leads to segmentation of the lower zone from the middle zone. In the final step, the few cases where region growing over-segments a single character are merged into a single segment by analyzing the metadata generated in the module described in Section 3.4. Our algorithm is tested on a database generated by [1] from 260 scene images and the experimental results establish satisfactory performance of our methods. The proposed system is robust to skewed and slanted text.

2. SEGMENTATION ARCHITECTURE

Here we describe the architecture of the system used to segment background and text and then segment the characters or their parts from text. The steps of segmenting the characters and parts of the characters from the input color image (I) are shown in the block diagram in Fig. 2. We will now discuss each of these steps in more detail in Section 3.

3. PROPOSED ALGORITHM

3.1 Text and Background Detection

In scene images some formidable characteristics are complex background with multiple colors, fonts with multiple colors, presence of border of fonts in different colors, effect of lightness, etc. Hence, converting the image into a binary image by applying popular global or local thresholding method cannot segment the text from the background properly. Often it leads to improper character or their part segmentation by our algorithm. Therefore, a combination of Otsu's thresholding method [14] and an unsupervised learning method is necessary to get better results in general. We applied unsupervised K-means clustering to cluster different color regions in an image. Often scene image texts are effected by varying lightness. To handle this lightness effect on an image we normalize the RGB values of an image before implementing K-means clustering. But we do not normalize those pixels where the pixel have near gray RGB values. So, for each pixel we check if,

$$(\max(r,g,b) - \min(r,g,b))/\max(r,g,b) > 0.2 \qquad (1)$$

The threshold value 0.2 is selected after extensive experiments to filter out the RGB values having near gray values. If Eq. 1 holds false then we do not perform RGB normalization on that pixel. We call this set of pixels G and we convert this RGB values to gray value and perform Otsu's thresholding to obtain a binary set. We term the complementary set of G as C and RGB normalization is carried out on C set according to the Eqs. 2 in order to remove the lightness effect from those pixels, keeping the color information intact.

$$\begin{aligned} p_{r_{norm}} &= p_r/(p_r+p_g+p_b) \\ p_{g_{norm}} &= p_g/(p_r+p_g+p_b) \\ p_{b_{norm}} &= p_b/(p_r+p_g+p_b) \end{aligned} \qquad (2)$$

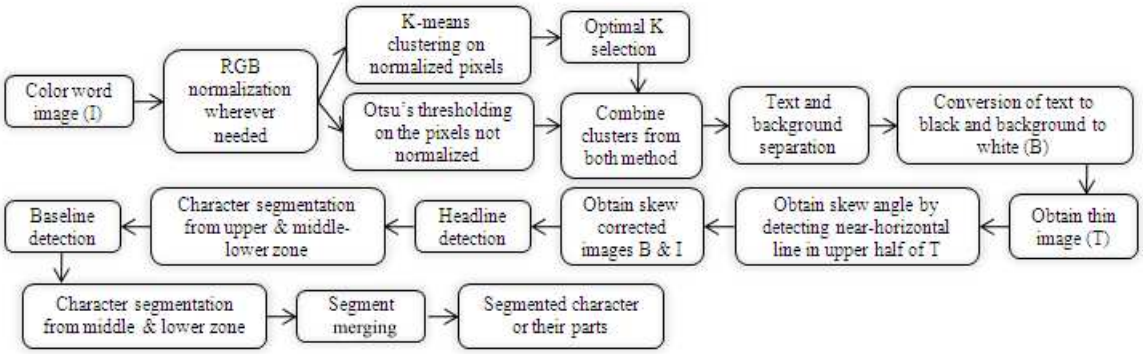


Figure 2: Block diagram of the algorithm.

Then we apply K-means clustering on C to obtain the text and the background separately. As our input image I is a bounding box of a localized text from the main image, it has been observed that generally the image does not contain more than 5 different color regions. Thus, we run K-means algorithm for K values ranging from 2 to 5, each for 50 iterations. We initialize the seed points of K clusters as follows. The RGB values of image are our data points. First, For K-means algorithm, we need $K(> 1)$ initial seed points. The first seed point s_1 is randomly selected from the set C. The second seed point s_2 is randomly selected from C such that $\|s_1 - s_2\| \geq 1/K$. Finally, the third seed point s_3 is selected so that $\|s_i - s_3\| \geq 1/K$, $for\ i = 1, 2$. We stop when K seed points are determined. After this K-means algorithm is employed on C. We compute the error of K-means clustering for each $K, K = 2, \ldots, 5$ by summing up within-cluster sums of point-to-seed squared distances and denote them by E_2, E_3, E_4 and E_5 respectively. We also computer the error value for the complete C set and denote it by E_1. Next, we calculate E_K/E_{K+1}, where $K = 1, \ldots, 4$. Note that, E_1/E_2 will be very high. We chose K when $E_K/E_{K+1} < 1.5$ for the first time. This is our optimal K value and we take the clusters obtained by this K value. Among these clusters, whichever contains the maximum number of boundary and penultimate boundary pixels RGB values of the image, gives the background color. In case of more than one cluster having approximately similar number of boundary and penultimate boundary RGB values we take any of them as the background. An extensive study shows that the strokes of scene texts are usually of uniform or almost uniform thickness. This helps us to determine which clusters form the text region. If we get more than one cluster of almost similar thickness then we merge them together. Thickness and uniformity of thickness is also calculated for the binary set obtained by Otsu's method. And clusters of similar thickness obtained from both methods are merged together. Thickness of individual clusters is obtained by using Distance Transform representation. For the clusters obtained by K-means algorithm we measure the distance of a data point of one cluster to the nearest data point of any other clusters. The uniformity of thickness of each cluster is measured by finding out the local maxima of each 3×3 sized window in the image obtained after distance transformation. Finally, we convert the text cluster to black and the background cluster to white and obtain binary image B. Fig. 3 illustrates how we separate set G and

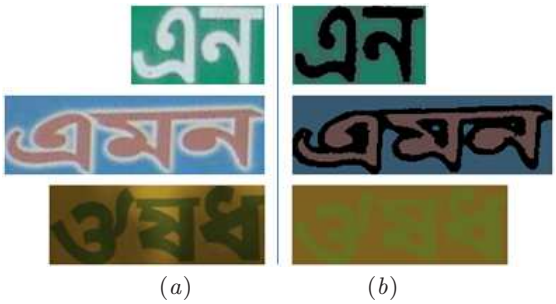


Figure 3: Illustration of proposed clustering method as a combination of Otsu's thresholding and K-means clustering. (a) Original images. (b) Clustering by our method. Black represents the set G and color represents the set C.

Figure 4: Binarization problems. (a) Original images. (b) Improper binary images obtained by Otsu's method. (c) Improved binary images obtained by our approach.

C to get proper binary image both in presence and absence of lightness variation. A few examples where our method gives better binary images than Otsu's method are shown in Fig. 4.

3.2 Skew Correction

Scene image texts are often skewed with respect to the horizontal axis (x-axis). We first correct the skew of a word for detection of its thick headline. In the literature, Hough transform had been used for skew correction of printed documents. In the present scenario, we failed to correct the skew of several word images with the help of Hough transform. The reason of failure is the thickness of the

characters in the word. To resolve this issue we apply the thinning algorithm [5] on B which generates the thinned image T. Since our input is a bounding box of a *Bangla* word, the headline of a word usually lies in the upper half. We apply Hough transform to obtain all line segments in the upper half of T and with slopes less than 65^0. If the length of the longest line segment among them is greater than an empirically selected threshold value, it is decided as the headline. If this headline is not parallel to the x-axis then its skew is corrected by rotating the word image accordingly. If the length of the longest line segment is less than the threshold value, we assume that the image either contains only numerals or texts without any headline. In such cases we move to the module for character segmentation (as described in Section 3.4). Otherwise, we detect the whole of the thick headline as discussed in Section 3.3.

3.3 Headline Detection

In *Bangla* texts, headline connects most save for a possible few characters in a word. So, in order to segment the characters or their parts in *Bangla* words we need to detect its thick headline and we achieve the same as follows. The projection profile is computed by row-wise sum of gray values for each row in the upper half of word image B. The heights of the projection profile are normalized between 0 and 1 ('1' corresponds to the longest projection profile). The row which contains the headline as obtained in the above (Section 3.2) is labelled as the spine of the headline. We scan the normalized projection profiles of successive rows in the upward direction starting from the spine and stop scanning when this value drops to less than a pre-defined threshold value. This row of the word image is considered as the upper boundary of the headline. Similarly, we scan these projection profile values downward starting from the spine and the row, for which this value drops to less than the same threshold value, is considered as the lower boundary of the headline.

3.4 Character Segmentation

In this module, we use the region growing method to extract the individual characters or their parts from the binarized and skew corrected word image B.

3.4.1 Region Growing

We locate the lowest and leftmost black pixel in B, and consider it as the seed point for our region growing module. The current segment is extracted using the standard region growing approach [9] based on 8-neighborhood. The stopping criteria for the implementation of region growing is either (i) we reach the upper or lower boundary of the thick headline, or (ii) we reach at a white pixel. The extraction of the current segment is continued until no pixel is left to visit satisfying the above.

3.4.2 Appending a Local Headline

This module is executed if a headline is detected in Section 3.2. Here, we append the part of the headline to the above extracted segment as follows. The top left and top right pixels of this segment lie on the lower boundary of the headline and the portion of the thick headline just above these two pixels are appended to the segment before its extraction. Finally, the pixels of B which form the extracted segment are converted to white and the module of Section 3.4.1 and Section 3.4.2 are repeated until there is no more black pixel in B.

A sample word image and its characters or their parts extracted as in the above are shown in Fig. 5.

Figure 5: A sample word image (top-left) and its segments obtained from the middle-lower (red) and the upper (green) zones during successive iterations. The yellow color refers to the local headline regions of respective segments.

3.4.3 Storing Metadata

Various information obtained during the above procedure such as the number of segments generated, location of headline (if exists), bounding box of each segment, normalized height and width of each segment lying in the middle-lower zone etc. are stored for their use during the following stage.

3.5 Baseline Detection

For baseline detection module we feed all the segments of the middle-lower zone which either hang from the headline or from immediate below (at most 0.2 times the height of the middle-lower region) the headline. Let n be the total number of such segments with respective heights $h_i, i \in \{1, 2, \ldots, n\}$ obtained by the region-growing algorithm of the previous step. In the present approach for baseline detection, the above heights are first normalized to h'_i, where $0 < h'_i \leq 10$, $\forall i$. Now, we find

$$h_{\min} = \min_i \{h'_i | h'_i > 6.0\} \tag{3}$$

Next, we find

$$h^* = \max_i \{h'_i | h'_i \geq h_{\min} \& h'_i < \lfloor h_{\min} \rfloor + 1\} \tag{4}$$

where $\lfloor x \rfloor$ denotes the floor value of x. The horizontal line through the bottom most pixel of the segment with normalized height h^* is the baseline obtained by our approach. The above empirical approach for the detection of baseline is developed through an extensive study of the word samples in our database of *Bangla* scene texts. The baseline detection approach presented in [3] obtains a straight line, which passes through the maximum number of lowermost pixels of the characters in a text line of scanned pages of printed documents. This method does not work for scene texts where we cannot assume the presence of similar text lines of the same font. Moreover, our input image contains one word and not a text line. Further details of the performance of these two approaches for baseline detection are explained in Fig. 6.

After detecting the baseline we can horizontally partition the lower zone of the character segments from the middle-lower zone.

3.6 Segment Merging

In some cases the character segmentation algorithm (described in Section 3.4) over-segments characters (as shown in Fig. 7) lying in the middle-lower zone of

(a) (b) (c)

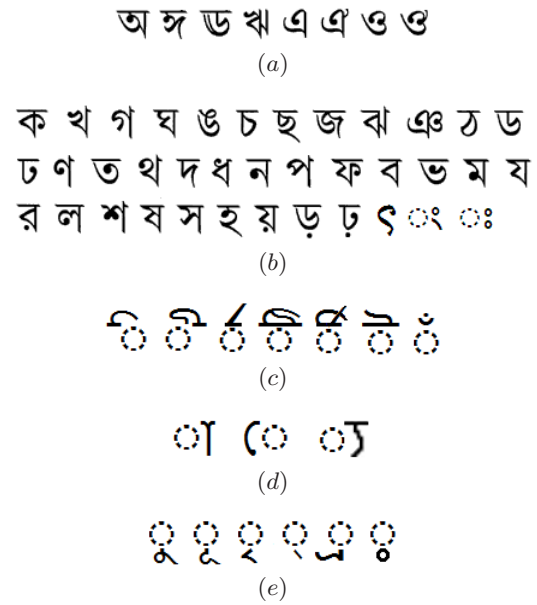
Figure 6: Three situations where an existing baseline detection algorithm [3] fails (shown in the left) but the proposed approach finds it successfully (shown in the right). The existing algorithm [3] obtains : (*a*) two baselines, (*b*) no distinct baseline, (*c*) a false baseline.

Figure 7: Examples of over-segmentations and results of their merging.

Bangla word. The present module identifies similar over-segmentations of characters and merges the relevant parts. The algorithm has the following three steps: (i) we identify segments which hang from the headline as described in Section 3.5 and has normalized heights smaller than 0.6, (ii) corresponding to each segment identified in step (i) above, we search for another segment to its right and having normalized height larger than 0.6 such that the bounding rectangles of the two segments either touches each other or overlaps or the smaller one is contained inside the larger one or these two are separated by at most a very small vertical gap, (iii) if a segment is identified by step (i) and a corresponding segment is identified by step (ii), then these two are merged to recover a character shape. Usually, the character shapes এ, গ, ণ, শ and ও get divided into two smaller segments by the region growing module of Section 3.4.1 and these are merged during the execution of the present module.

4. RESULTS

The proposed methodology of segmenting Bangla words in scene image into characters or their parts has been implemented using OpenCV library under the MS-Windows environment. For simulation purpose we considered an image database of 2460 Bangla words extracted from 260 scene images taken from the roads of West Bengal, India. Some results of our simulation are provided in the following two sections.



(a) (b) (c) (d) (e)

Figure 8: Basic characters, modifiers or their parts as obtained by our segmentation algorithm. (*a*) Basic vowels or their parts. (*b*) Basic consonants or their parts. (*c*) Modifiers or their parts and character parts occurring in the upper zone. (*d*) Modifiers or their parts occurring in the middle zone. (*e*) Modifiers, character parts and hasant(an auxiliary symbol) occurring in the lower zone. [Numerals and compound characters are not shown here].

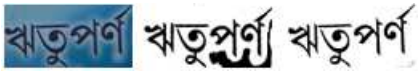
4.1 Clustering

The combination of Otsu's threshold selection and K-means clustering segments the text very finely which results in a high rate of character segmentation success. This gives very good results even for images with varying lightness effect. Otsu's method and K-means clustering together not only separates the background and text but can also segment out the artifacts that are not part of the text (example in Fig. 9(*b*)). Running the K-means clustering up to 5 clusters is sufficient for separating text and background regions for the type of images we use as input. Selection of optimal K value is a crucial step and our method selects it very efficiently.

4.2 Character Segmentation

Results of character segmentation by the proposed scheme on several samples of our image database are shown in Fig. 9(*a*). The text segmentation algorithm is able to handle and separate characters that horizontally overlap. Thus, slanted texts are segmented efficiently. Our dataset contains 2460 images, among which 2288 images have *Bangla* text and rest 172 images have *Bangla* numerals. The dataset contains 12380 symbols as described in Section 1 and 507 numerals from 146 classes (10 numerals, 8 vowels,

(a)



(b)

Figure 9: (a) Samples of characters or their parts as obtained by our segmentation algorithm. (b) original image, artifacts produced during binarization by Otsu's method and finally, our method segments only text from background and no artifact.

(a) (b) (c) (d) (e)

Figure 10: A few word images of our sample database for which our segmentation algorithm failed; (a) word contains touching characters, (b) image contains artifacts, (c) the word and its headline are curved, (d) varying brightness due to uneven reflection by the metallic substance of the word, (e) embossed text with poor contrast between foreground and background colors.

37 consonants, 7 symbols from upper zone, 3 symbols from middle zone, 6 symbols from lower zone, 75 compound characters). All these symbols excepting the numerals and compound characters are shown in Fig. 8.

Our system achieves 92.33% success with character segmentation and 96.64% success with numeral segmentation. These results are summarized in Table. 1

Table 1: Summary of segmentation results

Database(260 scene images)	Output Success
2460 word images	2282 words (92.8%)
12380 characters	11430 characters (92.33%)
507 numerals	405 numerals (96.64%)

About 7.2% of total images could not be or were partially segmented. These included touching characters, curved headline, artistic fonts, highly complex background, etc. About 61 touching characters could not be segmented. The algorithm cannot always segment the letter "জ" properly as its right part is sometimes not connected to the left part "ড" (it is observed that usually the right part is connected to headline few pixels away from the left part. The algorithm divides "জ" into two segments and unable to unify them). Presence of another character "গ" in text sometime results in improper baseline detection. Examples of some images where our system fails are illustrated in Fig. 10.

5. CONCLUSIONS

In this article, we proposed a novel approach for segmentation of Bangla words in scene images into characters or their parts. The proposed method of binarizing the color images of scene words by a combination of unsupervised K-means clustering and Otsu's thresholding method is novel and is shown to perform more efficiently than both of the global or local thresholding techniques. In many cases, our approach could handle the variation in brightness in the word image efficiently although there are a few failures in severe cases. Obtaining the optimal value for K is a challenging task and we achieved reasonable success in this respect too. Although the algorithm involves a skew correction module, slant correction is not needed for the proposed approach. However, it works miserably for curved texts. The algorithm not only segments a word into characters or their parts, but also segregates certain background artifacts. Texts from three zones are segmented separately so as to minimize the number of classes of basic characters. The proposed method for detection of baseline in scene words is novel and it should perform equally efficiently for ideal printed texts.

An additional advantage of the proposed segmentation approach lies in the fact that the metadata generated during the process may be useful for further analysis of the segments. For example, it will help in separating touching characters or find the association between segments lying in different zones.

6. REFERENCES

[1] U. Bhattacharya, S. K. Parui, and S. Mondal. Devanagari and bangla text extraction from natural scene images. *Proc. of Int. Conf. on Document Analysis and Recognition*, pages 171–175, 2009.

[2] R. G. Casey and E. Lecolinet. A survey of methods and strategies in character segmentation. *IEEE Transactions on Pattern Analysis and Machine Intelligence*, 18(7):690–706, 1996.

[3] B. B. Chaudhuri and U. Pal. A complete printed bangla ocr system. *Pattern Recognition*, 31(5):531–549, 1998.

[4] A. R. Chowdhury, U. Bhattacharya, and S. K. Parui. Text detection of two major indian scripts in natural scene images. *Proc. of CBDAR 2011*, pages 73–78, 2011.

[5] A. Datta and S. K. Parui. A robust parallel thinning algorithm for binary images. *Pattern Recognition*, 27:1181–1192, 1994.

[6] N. Ezaki, M. Bulacu, and L. Schomaker. Text detection from natural scene images, towards a system for visually impaired persons. *Proc. of 17th Int. Conf. on Patt. Recog.*, II:683–686, 2004.

[7] R. Ghoshal, A. Roy, T. K. Bhowmik, and S. K. Parui. Headline based text extraction from outdoor images. *PReMI*, pages 446–451, 2011.

[8] J. Gllavata, R. Ewerth, and B. Freisleben. Text detection in images based on unsupervised classification of high frequency wavelet coefficients. *Proc. of 17th Int. Conf. on Patt. Recog.*, 1:425–428, 2004.

[9] R. C. Gonzalez and R. E. Woods. *Digital Image Processing, 3rd Edition.* Pentice-Hall, Inc., Upper

Saddle River, New Jersey, 2006.

[10] K. Jung, K. I. Kim, and A. K. Jain. Text information extraction in images and video: a survey. *Pattern Recognition*, 37:977–997, 2004.

[11] J. Liang, D. Doermann, and H. Li. Camera based analysis of text and documents : a survey. *Int. Journ. on Doc. Anal. and Recog.*, 7:84–104, 2005.

[12] C. Mancas-Thillou. Character segmentation-by-recognition using log-gabor filters. *Proc. of ICPR*, 2:901–904, 2006.

[13] R. Nagy, A. Dicker, and K. Meyer-Wegener. Neocr: A configurable dataset for natural image text recognition. *Camera-Based Document Analysis and Recognition*, 7139:150–163, 2012.

[14] N. Otsu. A threshold selection method from gray-level histograms. *IEEE Transactions on Systems, Man, and Cybernetics*, 9(1):62–66, 1979.

[15] T. Saoi, H. Goto, and H. Kobayashi. Text detection in color scene images based on unsupervised clustering of multi- channel wavelet features. *Proc. of 8th Int. Conf. on Doc. Anal. and Recog.*, pages 690–694, 2005.